ПРОБЛЕМА ВЗАЄМНОГО ВИКЛЮЧЕННЯ ПАРАЛЕЛЬНИХ ПРОЦЕСІВ З СПІЛЬНОЮ ПАМ’ЯТТЮ

ПРОГРАМНЕ ВИРІШЕННЯ

Розширення «формули» Н.Вірта

- Згадуючи відому «формулу» Н.Вірта, можна тепер подати її для паралельних обчислень ще у більш розгорнутому вигляді:

Паралельні програми = Паралельні алгоритми + Структури даних + Взаємодії процесів.

- Особливість паралельного програмування, на відміну від послідовного, полягає саме у засобах для взаємодій, а поняття *процесу*, як одиниці роботи у паралельній системі, є ключовим для паралельного програмування.
- Характер взаємодій процесів залежить від парадигми паралельних обчислень, що зрештою визначається архітектурою мультипроцесорної системи.

Категорії взаємодії паралельних процесів

- Для мультипрограмних систем і мультипроцесорних систем з спільною пам'яттю взаємодії процесів поділяються *на дві категорії:*
- *взаємне виключення* (mutual exlusion) процесів – коли вони "змагаються" за спільний ресурс і можуть володіти чи мати доступи до нього тільки *поодинці*;
- *синхронізація* процесів, коли вони "співпрацюють" у вирішенні однієї задачі і повинні погоджувати, синхронізувати, свої дії при операціях над спільними об'єктами.

Проблема взаємного виключення: приклад (1)

- Для ілюстрації проблему розглянемо на прикладі паралельної системи з *двох процесів, що мають спільну змінну X*, в якій , наприклад, підраховується кількість звертань до вивідного пристрою. Кожний процес при такому звертанні мусить виконувати команду X:=X+1.
- Основними подіями, пов'язаними з такою операцією, є :
  - 1) виклик значення C’ змінної C із пам'яті на регістр процесора;
  - 2) нарощування величини C’ на 1 ;
  - 3) засилання нового значення C’ за адресою C.
- При одночасному виконанні команд X:=X+1 в обох процесах і *за відсутності будь-яких засобів управління доступом до спільних змінних* значення C’ є *недетермінованим*.

Проблема взаємного виключення: приклад (2)

- Отже, в попередньому прикладі, в залежності від порядку, в якому відбуваються зазначені вище події, можливими результатами спільного виконання команд може бути *нарощування значення X на l або на 2*.
- Така ситуація є типовою для паралельних процесів з спільною пам’яттю.
- Можна собі уявити, як важливо регулювати доступ до спільних змінних з боку паралельних процесів, коли, наприклад, виконуються дві паралельні транзакції при одночасному знятті з банківського рахунку двох сум: C: = C-sum1; C:=C-sum2.
- При такому недермінованому виконанні операцій зняття однієї з сум може залишитись *непоміченим*(!).

Попередні висновки

- Попередній приклад виявляє *дві основні проблеми*, пов'язані з паралелізмом виконання операцій над спільними змінними, а саме:

1. *Спільна пам’ять є джерелом недетермінізму обчислень*, спричиненого умовами оперативного середовища (наприклад, впливом операційної системи чи комп'ютерної мережі);
2. Результат обчислень залежить від *відносної швидкості* виконання процесів.

Поняття критичної секції

- Для уникнення неправильного (нефункціонального) виконання взаємно виключних процесів в теорії і практиці паралельних систем виникло поняття *критичної секції* (КС) (critical section) програми (процесу).
- Критична секція – це така ділянки програми, що:
  - 1) має виконуватись тільки одним процесом в кожний момент часу (*монопольний режим*);
  - 2) не може бути перервана виконанням іншого процесу (*безперервність виконання*).

Програмне вирішення проблеми взаємного виключення

- *Програмне вирішення* проблеми взаємного виключення – це спосіб вирішення цієї проблеми *тільки за рахунок (додаткових) спільних змінних*, без використання спеціальних типів і структур даних (таких як семафори)
- З часу усвідомлення проблеми взаємного виключення виявилось, що забезпечення вирішення цієї проблеми виключно програмними засобами взаємодії процесів через спільну пам'ять – дуже непроста задача.
- На шляху до викладення остаточного розв’язку (алгоритму Деккера) були зроблені *кілька проміжних кроків*, що допомогли виявити основні ідеї цього алгоритму.

Ідея «дверей»

- Це ідея розв'язку при тій умові, що 1) операція доступу (тобто перевірки умови, або запису нового значення спільної змінної) – є неділимою (неперервною у виконанні), і 2) з-поміж "одночасних“ доступів декількох процесів на оперативному рівні арбітром вибирається довільний порядок.
- var Door: (0,C); (* спільна змінна *)
- Door: =0; parbegin 1,2;
- (*процес 1 *) (*процес 2 *)
- 1: repeat 2: repeat
- until Door=0; until Door=0; (* *Одночасний вхід* ? *)
- КС: КС:
- Door:=0; Door:=0;
- goto 1; goto 2;
- parend.

Ідея «турнікету»

- Перетворимо тепер змінну Door в "чергу", що лінеаризує порядок доступу процесів до КС і спільної змінної (турнікет).
-
- var Turn:1..2; (*спільна змінна*)
- Turn:=1;
- 1: repeat 2: repeat
- until Turn=1; until Turn=2;
- КС КС
- Turn:=2; Turn:=1;
- goto 1; goto 2;
- (* *Можливість зупинки в КС* ? *)

Ідея «розподілених змінних»

- Значення in означає намір увійти або входження в КС. Значення out означає знаходження процесу поза межами КС.
- var P1,P2: (in, out); /*локальні змінні доступні для читання*.
- P1:=P2:= out;
- 1: P1:=in; 2: P2:=in;
- repeat repeat
- until P2=out ↔ until P1=out
- КС КС
- P1:=out; P2:=out;
- goto 1; goto 2;
- (* *Можливість зациклювання* ? *)

Алгоритм Деккера взаємовиключення двох процесів

```
var   P1,  P2: ( in, out);
        Turn: 1.. 2: Turn:=1;   P1:=out;   P2:=out;
     / * процес   1  */                          /*процес   2  */
1:  P1:=in;                                  2:  P2:=in
    if  P2 = in then                              if  P1 = in then
     begin  if  Turn = 2 then                      begin  if  Turn = 1 then
       begin  P1:=out                                 begin  P2:=out
            repeat                                        repeat
            until Turn = 1;                               until Turn = 2;
            P1:=in;                                        P2:=in;
       end;                                           end
       repeat until P2=out;                              repeat until P1=out;
     end                                            end
              КС                                          КС
     Turn:=2;                                       Turn:=1;
     P1:=out;                                        P2:=out;
     goto 1;                                         goto 2;
```

Особливості алгоритму Деккера

- До правильного вирішення веде поєднання двох підходів: спільної змінної Turn для розв'язання зациклення, і "розподілені" змінні для проблеми зупинки. Вперше таке вирішення було запропоновано голландським математиком Деккером і потім узагальнено іншим голландцем Дійкстрою у 60-х роках.
- Алгоритм Деккера забезпечує виникнення трьох умов:
  1. тільки один процес може знаходитися у своїй КС;
  2. затримки або зависання одного з процесів не впливають на можливість входження в КС інших процесів;
  3. рішення відносно входження процесу в КС приймається в скінченному інтервалі часу.
-